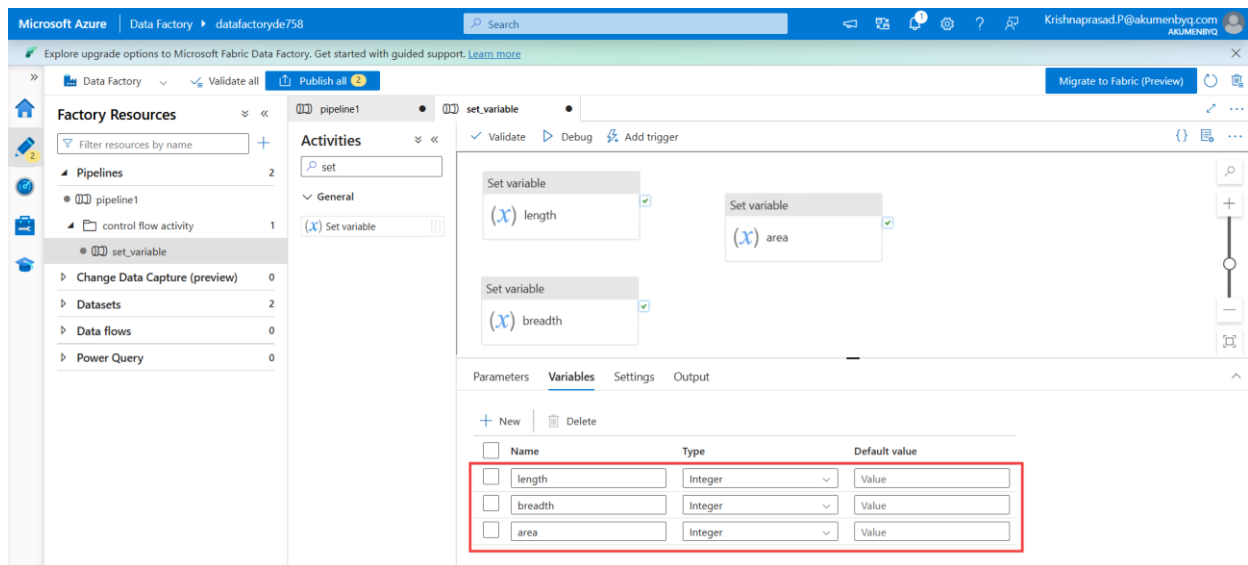


Azure Data Factory – Control Flow Activities

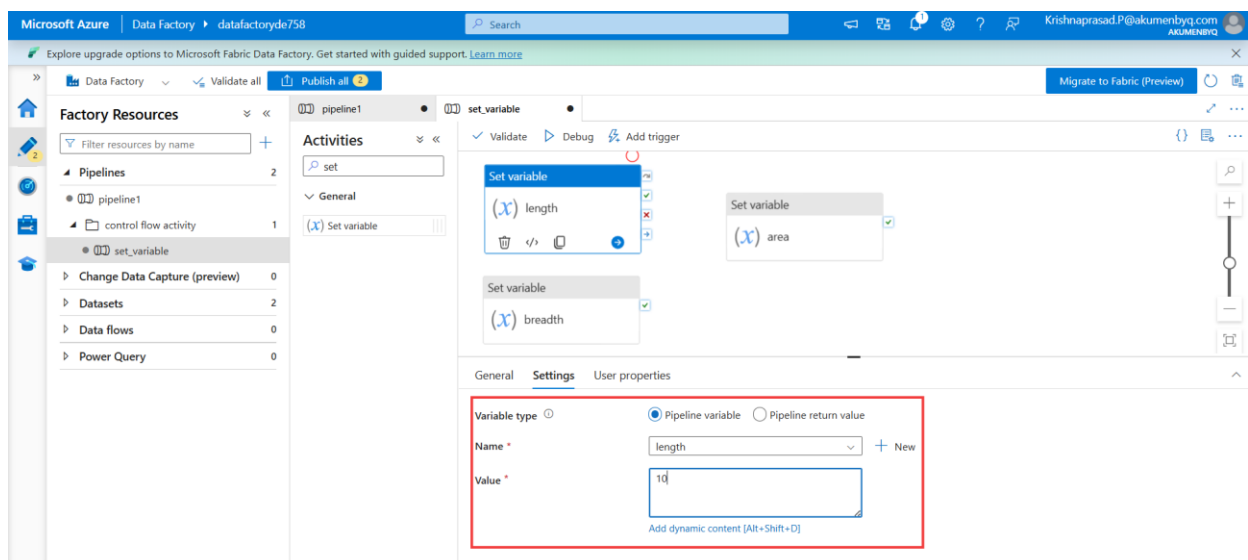
Step 1: Create Pipeline Variables

- Created a new pipeline named set_variable and added three pipeline variables — length, breadth and area — all of type Integer.

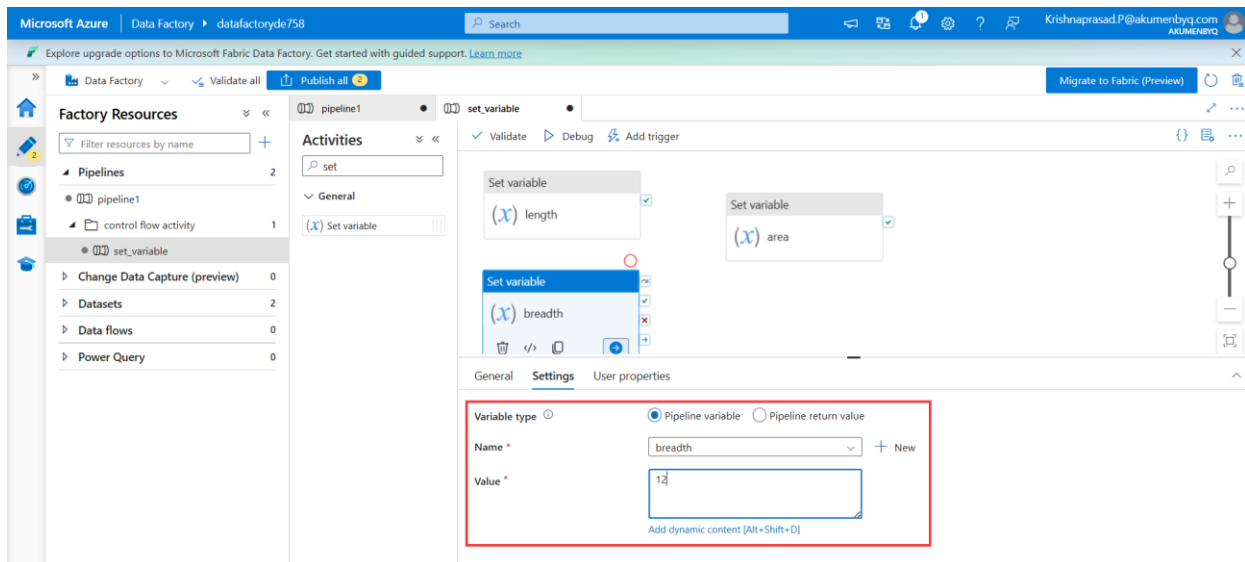


Step 2: Set the Length and Breadth Values

- Added a Set variable activity to assign the value 10 to length.

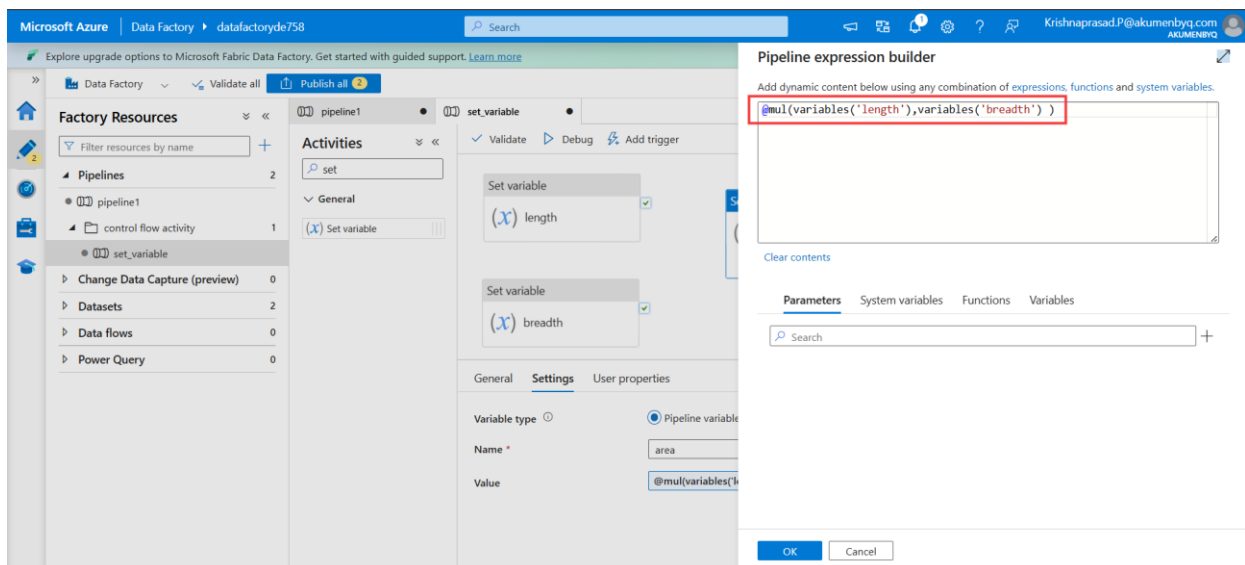


- Added a second Set variable activity to assign the value 12 to breadth.



Step 3: Calculate the Area

- Added a third Set variable activity for area, using the expression `@mul(variables('length'),variables('breadth'))` to multiply length by breadth.



Step 4: Debug and Verify

- Ran Debug — all three Set variable activities (length, breadth and area) succeeded.

The screenshot shows the Microsoft Azure Data Factory interface. On the left, the 'Factory Resources' pane lists 'Pipelines' with a count of 1. The main canvas displays a pipeline with three 'Set variable' activities: 'length', 'breadth', and 'area'. The 'area' activity is the final one in the sequence. Below the canvas, the 'Output' tab is selected, showing a table of activity results.

Activity name	Activity st...	Activit...	Run start	Duration	Integration run
area	✓ Succeeded	Set variable	8/18/2026, 6:58:09 PM	Less than 1s	
breadth	✓ Succeeded	Set variable	8/18/2026, 6:58:08 PM	Less than 1s	
length	✓ Succeeded	Set variable	8/18/2026, 6:58:08 PM	Less than 1s	

Step 5: Convert Variables to Pipeline Parameters

- Added length, breadth and area as pipeline parameters (type Int) so the values could be supplied at runtime instead of being hardcoded.

The screenshot shows the Microsoft Azure Data Factory interface with the 'Parameters' tab selected. The 'Parameters' pane at the bottom lists three parameters: 'length', 'breadth', and 'area', all of type 'Int'. The main canvas shows the same pipeline as before, but the 'length' variable's expression is now updated to reference the new parameter: '@pipeline().parameters.length'.

Name	Type	Default value
length	Int	Value
breadth	Int	Value
area	Int	Value

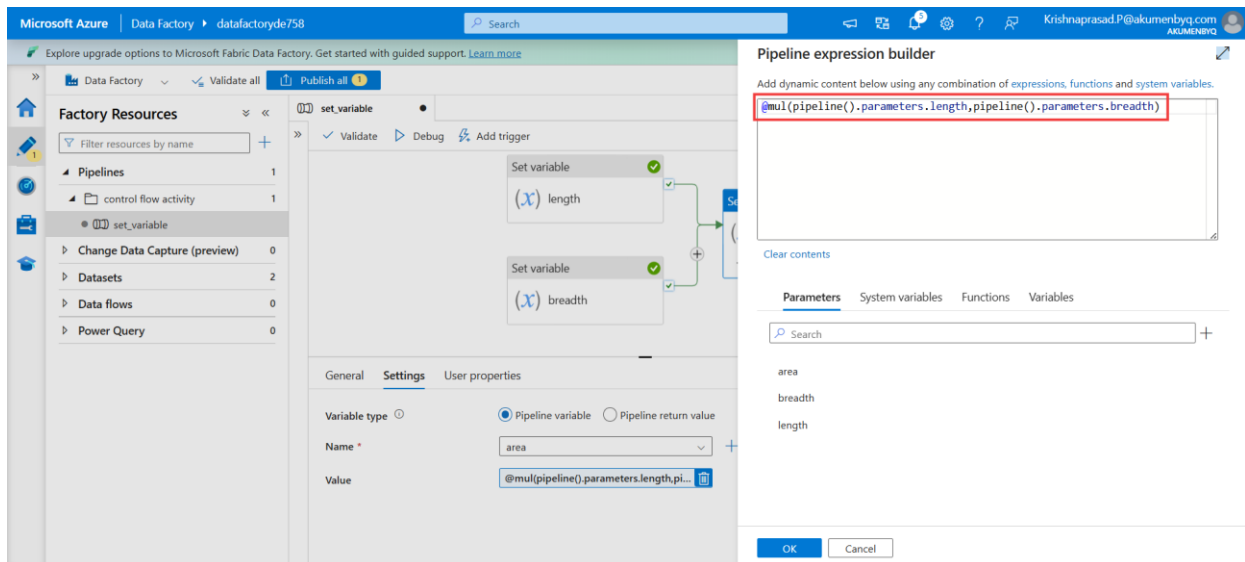
- Updated the length variable's expression to reference the new parameter: @pipeline().parameters.length.

The screenshot shows the Microsoft Azure Data Factory interface. On the left, the 'Factory Resources' pane lists 'Pipelines' with a count of 1. The main canvas displays a pipeline with two 'Set variable' activities. The first activity is for 'length' and the second is for 'breadth'. The 'Settings' tab for the 'length' activity is selected, showing 'Variable type' as 'Pipeline variable' and 'Name' as 'length'. On the right, the 'Pipeline expression builder' is open, showing the expression '@pipeline().parameters.length' in the text area. Below the text area, the 'Parameters' tab is selected, and a search bar is visible. The list of parameters includes 'area', 'breadth', and 'length'.

- Updated the breadth variable's expression the same way: @pipeline().parameters.breadth.

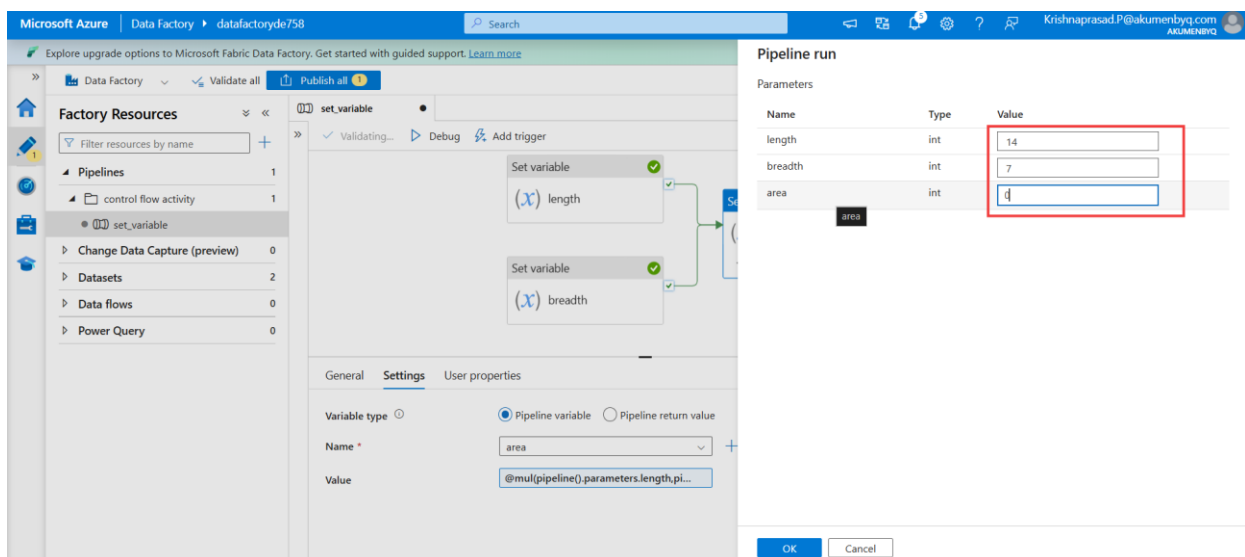
The screenshot shows the Microsoft Azure Data Factory interface. On the left, the 'Factory Resources' pane lists 'Pipelines' with a count of 1. The main canvas displays a pipeline with two 'Set variable' activities. The first activity is for 'length' and the second is for 'breadth'. The 'Settings' tab for the 'breadth' activity is selected, showing 'Variable type' as 'Pipeline variable' and 'Name' as 'breadth'. On the right, the 'Pipeline expression builder' is open, showing the expression '@pipeline().parameters.breadth' in the text area. Below the text area, the 'Parameters' tab is selected, and a search bar is visible. The list of parameters includes 'area', 'breadth', and 'length'.

- Updated the area expression to @mul(pipeline().parameters.length,pipeline().parameters.breadth).



Step 6: Run the Pipeline with Custom Parameters

- Triggered Debug, entering length = 14 and breadth = 7 in the Pipeline run dialog.



- All three activities — area, breadth and length — succeeded using the new parameter values.

Microsoft Azure | Data Factory | datafactoryde758

Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. [Learn more](#)

Factory Resources

- Pipelines: 1
 - control flow activity: 1
 - set_variable: 1
- Change Data Capture (preview): 0
- Datasets: 2
- Data flows: 0
- Power Query: 0

set_variable

Activities: Set variable (length), Set variable (breadth), Set variable (area)

Parameters Variables Settings **Output**

All status

Showing 1 - 3 of 3 items

Activity name	Activity st...	Activit...	Run start	Duration	Integration runtime	User prop...
area	Succeeded	Set variable	8/18/2026, 7:05:03 PM	Less than 1s		
breadth	Succeeded	Set variable	8/18/2026, 7:05:01 PM	Less than 1s		
length	Succeeded	Set variable	8/18/2026, 7:05:01 PM	Less than 1s		

Step 7: Build the Append Variable Pipeline

- Created a new pipeline named `append_variable` with an Append variable activity, and defined two array variables: `Marks` (default `["20","30","40","50"]`) and `Output`.

Microsoft Azure | Data Factory | datafactoryde758

Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. [Learn more](#)

Factory Resources

- Pipelines: 1
 - control flow: 1
 - append_variable: 1
- Change Data Capture (preview): 0
- Datasets: 2
- Data flows: 0
- Power Query: 0

append_variable

Activities: app

General: Append variable

Append variable

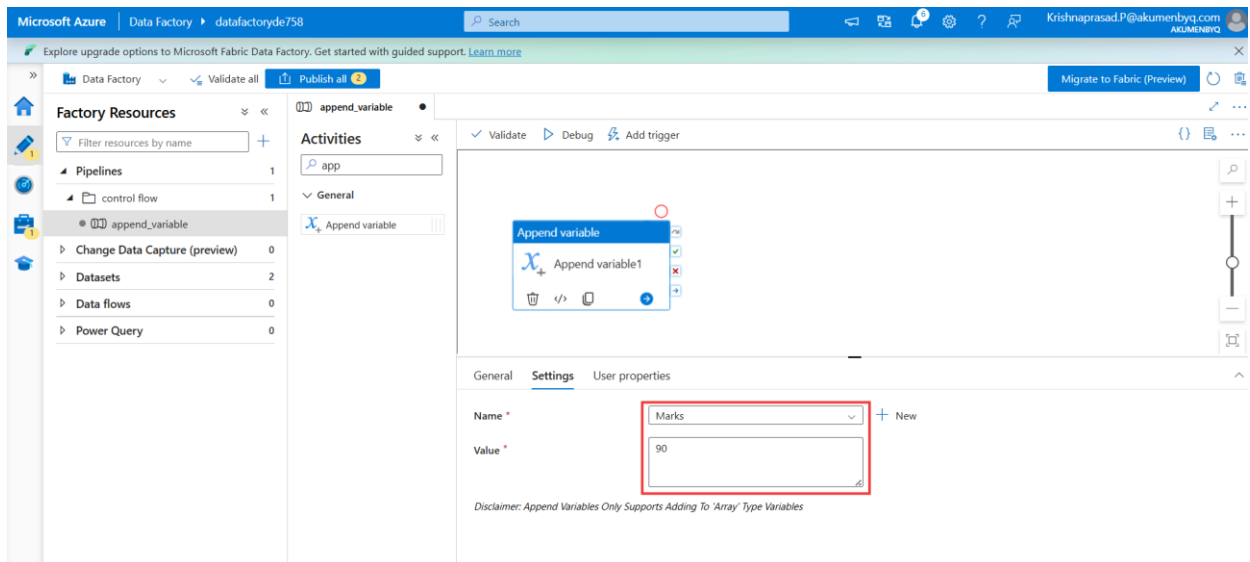
Parameters Variables Settings Output

+ New | Delete

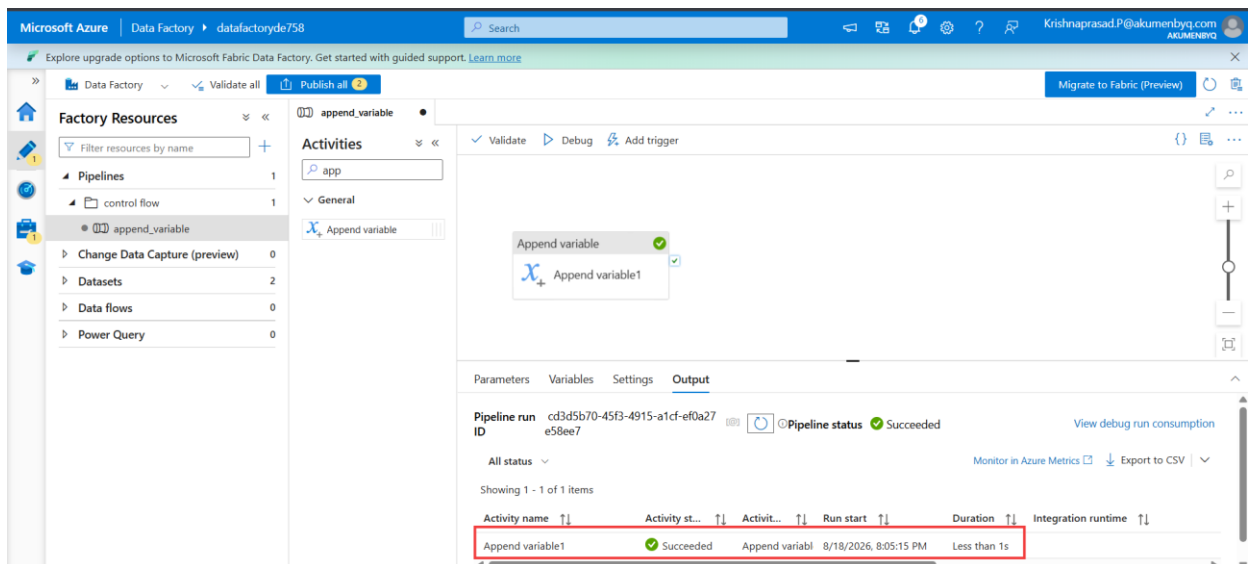
Name	Type	Default value
Marks	Array	["20","30","40","50"]
Output	Array	Value

Step 8: Append a Value to the Array

- Configured the Append variable activity to add the value 90 to the `Marks` array.



- Ran Debug — the Append variable1 activity succeeded.



Step 9: Copy the Array into the Output Variable

- Added a second activity, Set variable, after Append variable, setting Output to `@variables('Marks')`.

Microsoft Azure | Data Factory | datafactoryde758

Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. [Learn more](#)

Factory Resources

- Pipelines: 1
- control flow: 1
- append_variable: 1
- Change Data Capture (preview): 0
- Datasets: 2
- Data flows: 0
- Power Query: 0

Activities

- Synapse: Notebook
- Batch Service: Spark job definition, Custom
- General: (X) Set variable

Append variable

Set variable

Variable type: ☒ Pipeline variable ☐ Pipeline return value

Name: Output

Value: @variables('Marks')

- Ran Debug again — both Set variable1 and Append variable1 succeeded.

Microsoft Azure | Data Factory | datafactoryde758

Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. [Learn more](#)

Factory Resources

- Pipelines: 1
- control flow: 1
- append_variable: 1
- Change Data Capture (preview): 0
- Datasets: 2
- Data flows: 0
- Power Query: 0

Activities

- Synapse: Notebook
- Batch Service: Spark job definition, Custom
- General: (X) Set variable

Append variable

Set variable

Parameters Variables Settings Output

Pipeline run ID: 10/1/dcd/-/118-4411-a483-42350

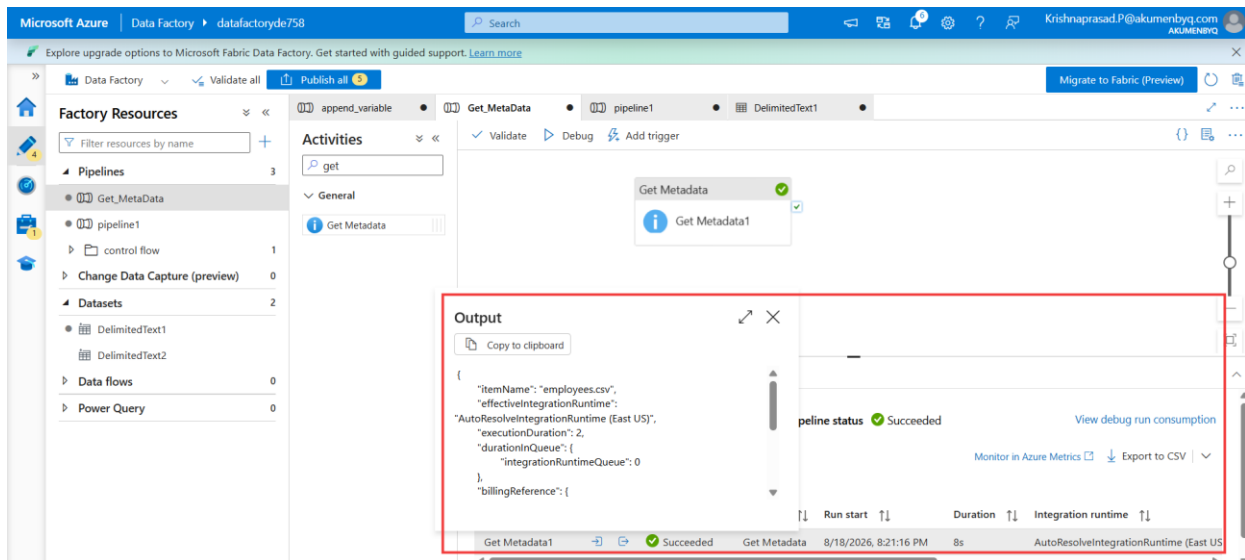
Pipeline status: Succeeded

Showing 1 - 2 of 2 items

Activity name	Activity st...	Activit...	Run start	Duration	Integration runtime
Set variable1	Succeeded	Set variable	8/18/2026, 8:07:39 PM	Less than 1s	
Append variable1	Succeeded	Append variabl	8/18/2026, 8:07:38 PM	Less than 1s	

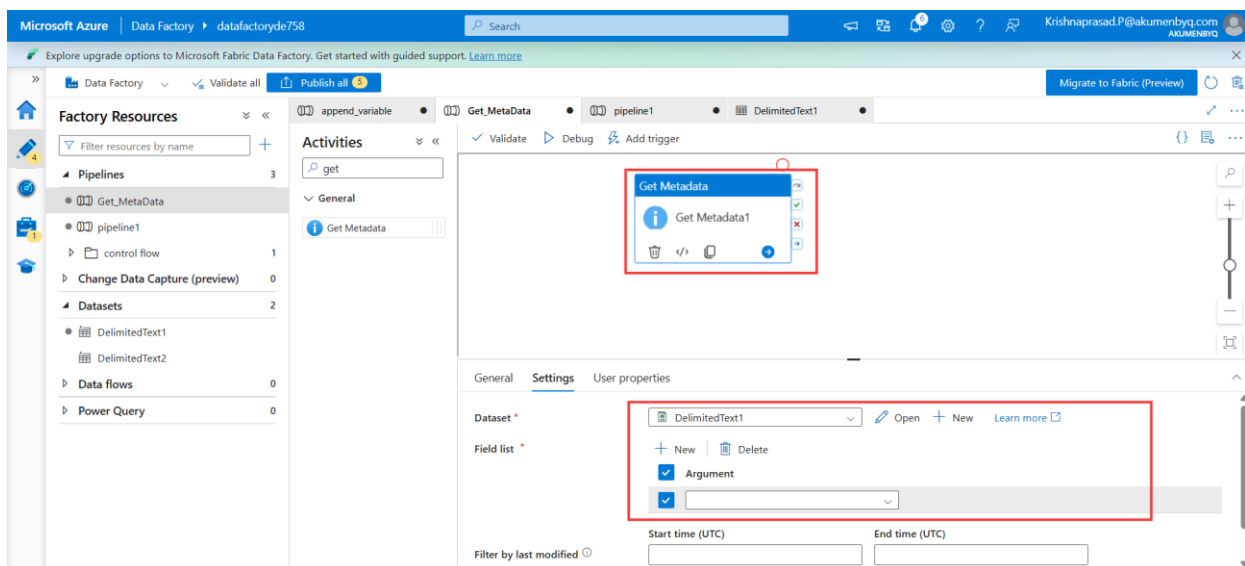
Step 10: Build the Get Metadata Pipeline

- Created a new pipeline named Get_MetaData with a Get Metadata activity, and ran Debug to retrieve file properties for the employees.csv dataset (DelimitedText1).



Step 11: Configure the Metadata Field List

- In the Get Metadata activity settings, set the dataset to DelimitedText1 and selected the Argument field to retrieve from the file.



Step 12: Capture the Pipeline Name

- Added a Set variable activity after Get Metadata, setting pipelineName to @pipeline().Pipeline to capture the current pipeline's name.

The screenshot shows the Microsoft Azure Data Factory interface. On the left, the 'Factory Resources' pane lists 'Pipelines' with 'Get_Metadata' and 'pipeline1'. The 'Activities' pane on the right shows 'Set variable' selected. The main canvas displays a workflow with 'Get Metadata' and 'Set variable' activities. The 'Set variable' activity is highlighted with a red box, showing its configuration: 'Variable type' is 'Pipeline variable', 'Name' is 'pipelineName', and 'Value' is '@pipeline().Pipeline'.

- Ran Debug — both Set variable1 and Get Metadata1 succeeded.

The screenshot shows the Microsoft Azure Data Factory interface with the 'Output' tab selected. It displays the execution results of the 'Set variable' and 'Get Metadata' activities. The 'Set variable' activity is highlighted with a red box, showing a 'Succeeded' status. The 'Get Metadata' activity is also highlighted with a red box, showing a 'Succeeded' status.

Activity name	Activity status	Activity status variable	Run start	Duration	Integration runtime
Set variable1	Succeeded	variable	8/18/2026, 8:30:47 PM	Less than 1s	
Get Metadata1	Succeeded	Get Metadata	8/18/2026, 8:30:35 PM	11s	AutoResolveIntegrationRuntime (East US)

Step 13: Capture the File Size

- Reconfigured the Set variable activity to instead store pipelinesize, using the expression `@activity('Get Metadata1').output.size` to capture the file's size from the Get Metadata output.

The screenshot shows the Microsoft Azure Data Factory interface. On the left, the 'Factory Resources' pane lists 'Pipelines' with 'pipeline1' selected. The 'Activities' pane shows 'Set variable' selected. The main canvas displays a workflow with a 'Get Metadata' activity (labeled 'Get Metadata1') connected to a 'Set variable' activity (labeled 'Set variable1'). The 'Set variable' activity is configured with the following settings:

- Variable type: ☒ Pipeline variable
- Name: pipelinesize
- Value: `@activity('Get Metadata1').output.size`

- Added pipelinesize as an Integer variable on the pipeline.

The screenshot shows the Microsoft Azure Data Factory interface. On the left, the 'Factory Resources' pane lists 'Pipelines' with 'pipeline1' selected. The 'Activities' pane shows 'Set variable' selected. The main canvas displays a workflow with a 'Get Metadata' activity (labeled 'Get Metadata1') connected to a 'Set variable' activity (labeled 'Set variable1'). The 'Set variable' activity is configured with the following settings:

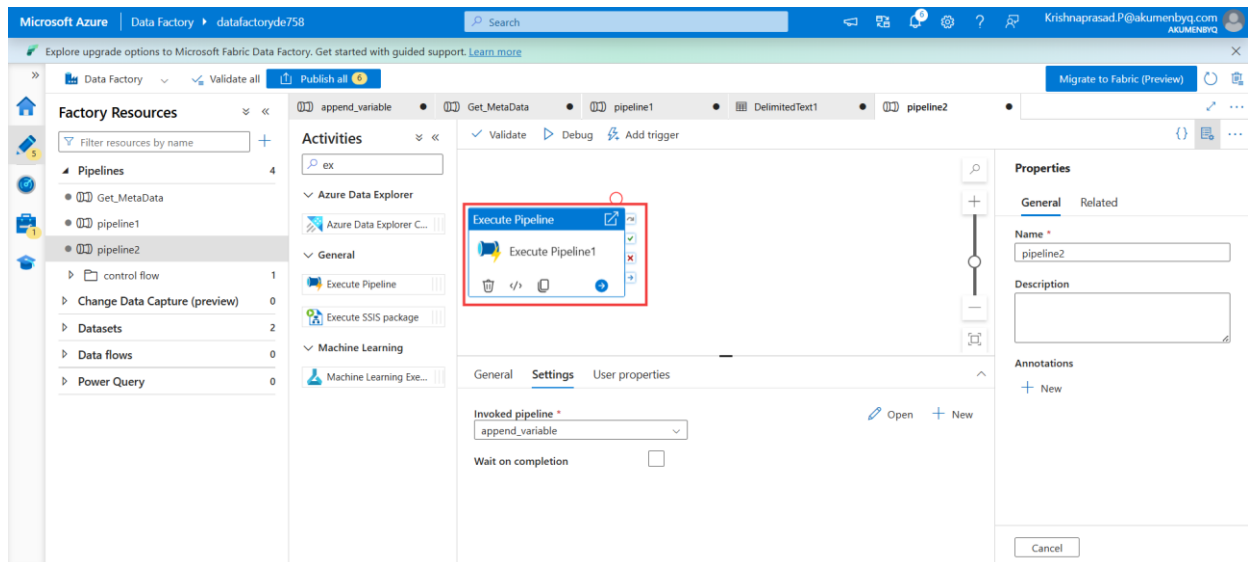
- Variable type: ☒ Pipeline variable
- Name: pipelinesize
- Value: `@activity('Get Metadata1').output.size`

Below the canvas, the 'Variables' tab is selected, showing a table with the following columns: Name, Type, and Default value.

Name	Type	Default value
pipelinesize	Integer	Value

Step 14: Orchestrate Pipelines with Execute Pipeline

- Created a new pipeline named pipeline2 with an Execute Pipeline activity, configured to invoke the append_variable pipeline.



- Ran Debug — the Execute Pipeline1 activity succeeded, confirming append_variable ran successfully as a child pipeline.

